Lab1 Doświadczenie w projekcie

IGOR BARCIKOWSKI

Realizowałem samodzielnie projekt platformy do skracania adresów URL. Założenie było proste: REST API w Javie 21 i Spring Boot, w którym użytkownik rejestruje się, potwierdza konto mailem, tworzy krótkie linki z datą ważności i sprawdza liczbę kliknięć. Do logowania z JWT doszedł Redis z ograniczaniem liczby zapytań, do niego klucze idempotentności, do wysyłki maili Kafka ze wzorcem Outbox, potem Prometheus i Grafana do obserwowania ruchu aplikacji, testy z Testcontainers, GitHub Actions z Qodaną i Trivy, a na końcu hosting w AWS. Było dwadzieścia funkcji, a potem do wszystkiego doszedł React.

Problemy zaczęły wychodzić dopiero przy uruchamianiu systemu w chmurze. Lokalnie wiadomości obsługiwała Kafka, ale na produkcji musiałem użyć Amazon SQS, więc w praktyce utrzymywałem dwie wersje tego samego mechanizmu (lokalnie oraz w chmurze). Automatyczny deployment i sekrety w Secret Managerze również został zaimplementowany. Testy bezpieczeństwa, kontraktowe i wydajnościowe ograniczały się do tego co sam zaobserwowałem mimo że bezpieczeństwo było jednym z głównych założeń systemu. Nikt tych błędów nie wychwycił, bo nikt poza mną nie czytał ani kodu, ani dokumentacji.

Wnioski: Przy projekcie, który nie ma zespołu ani klienta, to autor musi pilnować zakresu, bo nikt inny tego nie zrobi. Na początku trzeba zdefiniować minimalną wersję produktu i kryterium zakończenia, a pomysły "na przyszłość" odkładać do osobnego backlogu, zamiast dopisywać je do planu bieżącego. Elementy typu "wdrożenie" i "środowisko produkcyjne" zawsze zawierają więcej pracy niż wynika z pierwszego szacunku, więc konfigurację środowisk i sekretów należy zaplanować przed pierwszym wdrożeniem, a nie po nim. Dokumentacja powinna być aktualizowana razem ze zmianą kodu, w tej samej czynności, inaczej szybko przestaje być prawdziwa. Role w projekcie warto rozdzielać nawet wtedy, gdy pracuje jedna osoba: automatyczne kontrole (Qodana, Trivy, Dependabot) i przegląd własnego repozytorium "cudzymi oczami" po kilku dniach przerwy wyłapałyby większość opisanych błędów. Wreszcie, jakość trzeba weryfikować, a nie deklarować: jeśli system ma być bezpieczny, testy bezpieczeństwa należą do zakresu podstawowego, a nie do rzeczy odkładanych na później.